AGG 2.0 – New Features for Specifying and Analyzing Algebraic Graph Transformations

Olga Runge[1], Claudia Ermel[1], and Gabriele Taentzer[2]

[1] Technische Universität Berlin, Germany
{olga.runge,claudia.ermel}@tu-berlin.de
[2] Philipps-Universität Marburg, Germany
taentzer@informatik.uni-marburg.de

Abstract. The integrated development environment AGG supports the specification of algebraic graph transformation systems based on attributed, typed graphs with node type inheritance, graph rules with application conditions, and graph constraints. It offers several analysis techniques for graph transformation systems including graph parsing, consistency checking of graphs as well as conflict and dependency detection in transformations by critical pair analysis of graph rules, an important instrument to support the confluence check of graph transformation systems. AGG 2.0 includes various new features added over the past two years. It supports the specification of complex control structures for rule application comprising the definition of control and object flow for rule sequences and nested application conditions. Furthermore, new possibilities for constructing rules from existing ones (e.g., inverse, minimal, amalgamated, and concurrent rules) and for more flexible usability of critical pair analyses have been realized.

Keywords: graph transformation tool, AGG 2.0.

1 Introduction

AGG [15,13] is a well-established tool environment for algebraic graph transformation systems, developed and extended over the past 20 years. Graphs in AGG are defined by a type graph with node type inheritance and may be attributed by any kind of Java objects. Graph transformations can be equipped with arbitrary computations on these Java objects described by Java expressions.

The AGG environment consists of a graphical user interface comprising several visual editors, an interpreter, and a set of validation tools. The interpreter allows the stepwise transformation of graphs as well as rule applications as long as possible. AGG supports several kinds of validations which comprise graph parsing, consistency checking of graphs, applicability checking of rule sequences, and conflict and dependency detection by critical pair analysis of graph rules. Applications of AGG include graph and rule-based modeling of software, validation of system properties by assigning an operational semantics to some system model, graph transformation-based evolution of software, and the definition of visual languages by graph grammars.

A. Schürr, D. Varró, and G. Varró (Eds.): AGTIVE 2011, LNCS 7233, pp. 81–88, 2012.
© Springer-Verlag Berlin Heidelberg 2012

Model transformations have recently been identified as a key subject in model-driven development (MDD). Graph transformations offer useful concepts for MDD, while the software engineering community can generate interesting challenges for the graph transformation community. In the past two years, those new challenges also lead to the augmentation of AGG by new tool features. In this paper, we describe some of those challenges, the formal approaches developed to solve them, and the impact they had on recent developments of new features of AGG, leading to AGG 2.0.

2 Rule Application Control

Application Conditions. For graph transformation rules, well-known negative or positive application conditions (NACs, PACs) may be used that forbid or require a certain structure to be present in the graph for the rule to be applied. As a generalization, *application conditions* (introduced as *nested application conditions* in [9]) further enhance the expressiveness of graph transformations by providing a more powerful mechanism to control rule applications (see [8] for an extensive case study).

An *application condition* ac of rule $r : L \to R$ is of the form *true* or $\exists(a, c)$ where $a : L \to C$ is a graph morphism from L to a condition graph C, and c is a condition over C. Application conditions may be nested, negated,[1] quantified by FORALL and combined by using the logical connectors AND and OR. Given application condition ac, a match $m : L \to G$ satisfies ac, written $m \models ac$, if $ac = true$. A match $m : L \to G$ satisfies condition $ac = \exists(a, c)$ if there is an injective graph morphism $q : C \to G$ such that $q \circ a = m$ and q satisfies c. The satisfaction of conditions is extended to Boolean conditions in the usual way. A rule $L \to R$ is applicable only if the application condition ac is satisfied for its match $m : L \to G$.

An example is shown in Figure 1, where the activation of an elementary Petri net transition is checked: Rule ActivationCheck sets the transition attribute isActivated to true if two conditions, called PreCond and PostCond are satisfied: PostCond is the NAC shown in Figure 1 (b) which forbids the existence of a marked place in the transition's post-domain, and PreCond is a nested application condition shown in Figure 1 (c) which requires that on each place in the transition's pre-domain, there must be one token. Note that this condition cannot be expressed by using simple NACs or PACs. Figure 1 (a) shows the context menu entries for generating general application conditions (GACs) in AGG 2.0.

Object Flow for Rule Sequences. Object flow between rules has been defined in [11] as partial rule dependencies relating nodes of the RHS of one rule to (type-compatible) nodes of the LHS of a (not necessarily direct) subsequent rule in a given rule sequence. Object flow thus enhances the expressiveness of graph transformation systems and reduces the match finding effort.

[1] An application condition of the form $\neg\exists a$ is called *negative application condition.*

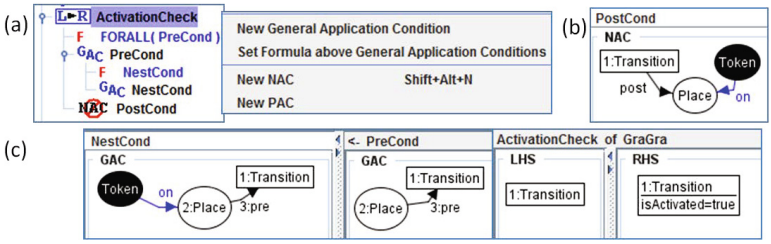

Fig. 1. Nested application conditions in AGG

In AGG 2.0, object flow can be defined between subsequent rules in a rule sequence, and the rule sequence can be applied to a given graph respecting the object flow. An example is the definition of a Petri net transition firing step by the rule sequence (ActivationCheck, RemovePre*, AddPost*, DeActivate) with object flow. The sequence defines that rule ActivationCheck (see Figure 1) is applied once, followed by rules RemovePre (removing a token from a pre-domain place), AddPost (adding a token to a post-domain place), which are shown in Figure 2, and DeActivate (setting the transition attribute isActivated back to false).

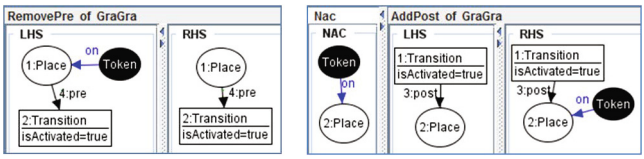

Fig. 2. Rules RemovePre and AddPost for Petri net firing

In the sequence, rules RemovePre and AddPost are applied as long as possible (denoted by "*"), and rule DeActivate is applied once. To restrict the application of the rule sequence to exactly one transition, we need to express that the transition in the matches of all rules is the same. This is done by defining the object flow, e.g., by mapping the transition from the RHS of rule ActivationCheck to the LHSs of rules RemovePre and AddPost as shown in Figure 3.

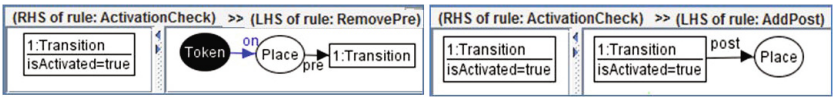

Fig. 3. Object flow definition for rule sequences in AGG

An example of a firing step by applying the rule sequence with object flow is shown in Figure 4, where the left transition in the net was selected, found activated and fired.

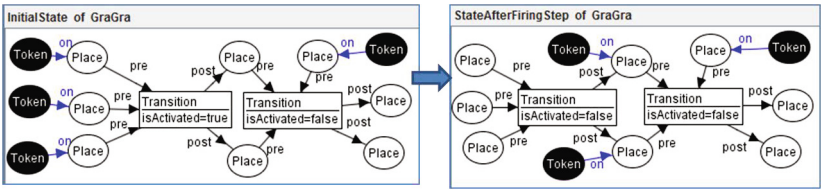

Fig. 4. Transition firing step resulting from applying a rule sequence with object flow

3 Constructing Rules from Existing Ones

Obviously, the definition of a rule sequence with object flow is neither simple nor very intuitive to model Petri net transition firing steps for transitions with an arbitrary number of pre- and post-domain places. A way closer to the inherent Petri net semantics (a transition removes the tokens from all of its pre-domain places and adds tokens to all of its post-domain places in *one atomic* step) would be to construct a *single* rule modelling the firing behaviour of a transition. AGG 2.0 supports various ways to automatic rule construction.

Construction of Concurrent Rules. A concurrent rule summarizes a given rule sequence in one equivalent rule [5,12]. In a nutshell, a concurrent rule $p{*_E}q = (L \leftarrow K \rightarrow R)$ is constructed from two rules $p = (L_p \leftarrow K_p \rightarrow R_p)$ and $q = (L_q \leftarrow K_q \rightarrow R_q)$ that may be sequentially dependent via an overlapping graph E by the diagram to the right, where (5) is pullback (intersection of graphs C_p and C_q in E), and all remaining squares are pushouts. (A pushout is a union of two graphs over an interface graph.) The concurrent rule $(p_1{*}\ldots{*}p_n)_E$ with $E = (E_1, \ldots, E_{n-1})$ for a sequence is constructed in an iterated way by $(p_1 * \ldots * p_n)_E = p_1 *_{E_1} p_2 *_{E_2} \ldots *_{E_{n-1}} p_n$. In AGG 2.0, we can construct a concurrent rule from a rule sequence with the following options concerning the overlappings of one rule's RHS with the succeeding rule's LHS: (1) compute maximal overlappings according to rule dependencies; (2) compute all possible overlappings (this usually yields a large number of concurrent rules); (3) compute overlappings based on the previously defined object flow between given rules; (4) compute the parallel rule (no overlapping), where rule graphs are disjointly unified, with NACs constructed according to [12]. As example, Figure 5 shows the concurrent rule constructed from rule sequence (ActivationCheck, RemovePre(3), AddPost(2), DeActivate).[2] Since an object flow is defined for this sequence, we choose option (3) *by object flow* for computing rule overlappings. Constraints on the type graph (e.g., "a Token node is connected to exactly one Place node") prevent the generation of unnecessary NACs.

$$\begin{tikzcd}
L_p \arrow[d] & K_p \arrow[l] \arrow[r] \arrow[d] \arrow[dr, phantom, "(1)"] & R_p \arrow[d] \arrow[dr, "e_p"] \arrow[dr, phantom, "(2)"] & & L_q \arrow[dl, "e_q"'] & K_q \arrow[l] \arrow[r] \arrow[d] \arrow[dl, phantom, "(3)"] & R_q \arrow[d] \arrow[dl, phantom, "(4)"] \\
L & C_p \arrow[l] \arrow[rr] & & E & & C_q \arrow[ll] \arrow[r] & R \\
& & & K \arrow[ull] \arrow[urr] \arrow[u, phantom, "(5)"] & & &
\end{tikzcd}$$

[2] Note that a concurrent rule can be constructed only for a finite rule sequence.

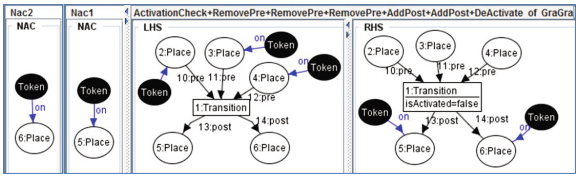

Fig. 5. Concurrent rule generated from a rule sequence in AGG

Construction of Amalgamated Rules. If a set of rules $p_1, \ldots, p_n$ share a common subrule p_0, a set of "amalgamable" transformations $G \overset{(p_i,m_i)}{\Longrightarrow} G_i$ $(1 \leq i \leq n)$ leads to an amalgamated transformation $G \overset{\tilde{p},\tilde{m}}{\Longrightarrow} H$ via the amalgamated rule $\tilde{p} = p_1 +_{p_0} \ldots +_{p_0} p_n$ constructed as gluing of $p_1, \ldots, p_n$ along p_0. We call p_0 *kernel rule*, and $p_1, \ldots, p_n$ *multi-rules*. A kernel rule together with its embeddings in several multi-rules is called *interaction scheme*. An amalgamated transformation $G \overset{\tilde{p}}{\Longrightarrow} H$ is a transformation via the amalgamated rule $\tilde{p}$ [8].

This concept is very useful to specify ∀-quantified operations on recurring graph patterns (e.g., in model refactorings). The effect is that the kernel rule is applied only once while multi-rules are applied as often as suitable matches are found. For formal details on amalgamation and a large case study, see [8].

Figure 6 shows the interaction scheme with a kernel rule and two multi-rules specifying the firing of a Petri net transition with arbitrary many pre- and post-domain places. The kernel rule has the same application condition as rule ActivationCheck (see Figure 1). Note that we do not need the isActivated flag any more because the check and the complete firing step is performed by a single application of the amalgamated rule. The amalgamated rule constructed for example along a match to an activated transition with three pre- and two post-domain places is similar to the rule in Figure 5 but has no NACs because the match into the host graph is predefined by construction.

Construction of Inverse Rules. For a given rule, the inverse rule has LHS and RHS exchanged. Moreover, application conditions are shifted over the rule. Figure 7 shows the inverse rule of rule AddPost (see Figure 2), where an existing token is removed. The shifted NAC requires that there is exactly one token on the place for the rule to be applicable.

Construction of Minimal Rules. A new challenge from MDD comes from the field of *model versioning*, where the new notion of *graph modification* [14], a span $G \leftarrow D \rightarrow H$ has been established to formalize model differences for visual models. Based on graph modifications, so-called *minimal rules* may be extracted from a given span to exploit conflict detection techniques for rules. A minimal rule comprises the effects of a given rule in a minimal context. Via context menu, AGG 2.0 supports the extraction of a minimal rule from a selected rule (interpreted as graph modification). For example, the minimal rule of the rule in Figure 5 does not contain the arcs connecting the places and transitions, since

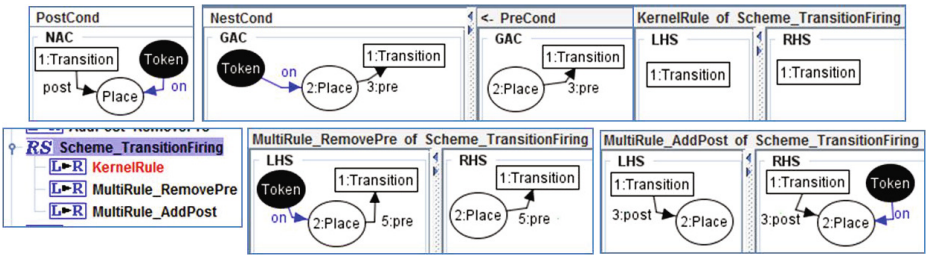

Fig. 6. Interaction scheme defining a transition firing step in Petri nets

Fig. 7. Inverse rule of Rule AddPost

these arcs are not changed by the rule. It contains the place nodes (because edges connected to them are deleted or generated), the token nodes (either deleted or generated) and the transition node (its attribute is changed).

4 Tuning the Critical Pair Analysis

The critical pair analysis (CPA) checks rule pairs for conflicts of rule applications in a minimal context. An example for a conflict in a minimal context is shown in Figure 8, where the classical *forward conflict* in Petri nets is detected when analyzing the rule pair (RemovePre, RemovePre). As indicated in the conflict view, we have a *delete-use conflict* since two transitions need to remove the same token from their common pre-domain place.

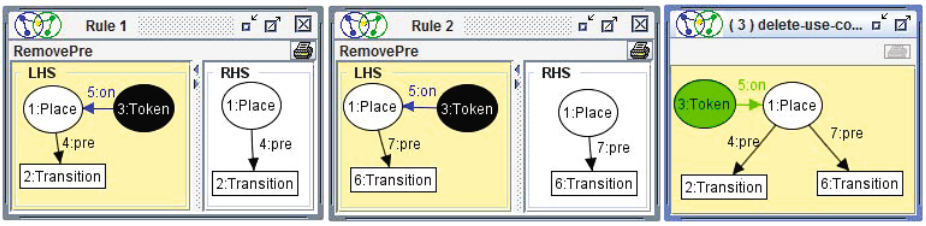

Fig. 8. CPA of rule pair (RemovePre, RemovePre), detecting a forward conflict

Selection of rules for CPA. So far, the CPA could be evoked on a graph grammar, yielding the critical pairs for each possible rule pair of the grammar. AGG 2.0 provides free selection of rule sets to be analyzed. This feature has shown to be very convenient, e.g., for a case study, where self-healing systems are modeled by several rule sets (normal system behaviour rules, context-changing rules, repair rules). These sets are analyzed for conflicts with each other [6].

Modularization of a model into different sub-grammars. In AGG 2.0, rule sets may be imported into an existing grammar (provided the type graph of the imported grammar is a subgraph of the type graph of the importing grammar). This supports modularization of a model without destroying the possibility to analyze the complete system.

Interrupt and resume running CPA. This usability feature is very handy for complex computations which may take some time. A partial CPA result may be stored and reloaded in AGG 2.0.

Generation of Filter NACs. If during CPA, critical pairs are found that are analyzed by the user as not causing real conflicts, additional NACs for these rules may be generated automatically that contain the critical overlapping region. A new CPA of the rules together with the new *Filter NACs* does not show the previous critical pair anymore [10].

5 Related Work and Conclusions

AGG is one of the standard graph transformation tools implementing the algebraic approach as presented in [5]. Other graph transformation tools, such as Fujaba [7], ViaTra [4], VMTS [3], GrGen [16], and Groove [17], implement different kinds of graph transformation approaches. Some kinds of rule application control structures are offered by all of these tools, e.g., Fujaba uses story diagrams, a kind of activity diagrams. Groove also supports nested application conditions as well as universal quantification using amalgamation.

Concerning the verification of graph transformation systems, VIATRA and Groove concentrate on some kind of model checking, while AGG is the only tool that consequently implements the theoretical results available for algebraic graph transformation. These results are mainly concerned with conflict and dependency detection of rules and static applicability checks for rule sequences. The analysis features offered by AGG are also used by our graph transformation-based tools Tiger (a tool for generating visual editor and simulator plug-ins in Eclipse [2]) and Henshin (an EMF model transformation engine [1]).

AGG 2.0 extends the existing features now coherently with support for application conditions and object flow, and for automatic construction of amalgamated, concurrent, inverse and minimal rules. Moreover, the critical pair analysis has become more usable due to experiences made in several case studies.

References

1. Arendt, T., Biermann, E., Jurack, S., Krause, C., Taentzer, G.: Henshin: Advanced Concepts and Tools for In-Place EMF Model Transformations. In: Petriu, D.C., Rouquette, N., Haugen, Ø. (eds.) MODELS 2010, Part I. LNCS, vol. 6394, pp. 121–135. Springer, Heidelberg (2010)
2. Biermann, E., Ehrig, K., Ermel, C., Hurrelmann, J.: Generation of simulation views for domain specific modeling languages based on the Eclipse Modeling Framework. In: Automated Software Engineering (ASE 2009), pp. 625–629. IEEE Press (2009)
3. Budapest University of Technology and Economics: Visual Modeling and Transformation System (VMTS) (2010), `http://www.aut.bme.hu/Portal/Vmts.aspx`
4. Eclipse Consortium: VIATRA2 (Visual Automated Model Transformations) Framework (2011), `http://www.eclipse.org/gmt/VIATRA2/`
5. Ehrig, H., Ehrig, K., Prange, U., Taentzer, G.: Fundamentals of Algebraic Graph Transformation. EATCS Monographs in Theoretical Computer Science. Springer (2006)
6. Ehrig, H., Ermel, C., Runge, O., Bucchiarone, A., Pelliccione, P.: Formal Analysis and Verification of Self-Healing Systems. In: Rosenblum, D.S., Taentzer, G. (eds.) FASE 2010. LNCS, vol. 6013, pp. 139–153. Springer, Heidelberg (2010)
7. Fujaba Development Group: Fujaba Tool Suite (2011), `http://www.fujaba.de/`
8. Golas, U., Biermann, E., Ehrig, H., Ermel, C.: A visual interpreter semantics for statecharts based on amalgamated graph transformation. In: Echahed, R., Habel, A., Mosbah, M. (eds.) Selected Papers of International Workshop on Graph Computation Models (GCM 2010). ECEASST, vol. 39 (2011)
9. Habel, A., Pennemann, K.H.: Correctness of high-level transformation systems relative to nested conditions. Mathematical Structures in Computer Science 19, 1–52 (2009)
10. Hermann, F., Ehrig, H., Golas, U., Orejas, F.: Efficient analysis and execution of correct and complete model transformations based on triple graph grammars. In: Proc. Int. Workshop on Model Driven Interoperability, pp. 22–31. ACM (2010)
11. Jurack, S., Lambers, L., Mehner, K., Taentzer, G., Wierse, G.: Object Flow Definition for Refined Activity Diagrams. In: Chechik, M., Wirsing, M. (eds.) FASE 2009. LNCS, vol. 5503, pp. 49–63. Springer, Heidelberg (2009)
12. Lambers, L.: Certifying Rule-Based Models using Graph Transformation. Ph.D. thesis, Technische Universität Berlin (2009)
13. Taentzer, G.: AGG: A Graph Transformation Environment for Modeling and Validation of Software. In: Pfaltz, J.L., Nagl, M., Böhlen, B. (eds.) AGTIVE 2003. LNCS, vol. 3062, pp. 446–453. Springer, Heidelberg (2004)
14. Taentzer, G., Ermel, C., Langer, P., Wimmer, M.: Conflict Detection for Model Versioning Based on Graph Modifications. In: Ehrig, H., Rensink, A., Rozenberg, G., Schürr, A. (eds.) ICGT 2010. LNCS, vol. 6372, pp. 171–186. Springer, Heidelberg (2010)
15. TFS-Group, TU Berlin: AGG 2.0 (2011), `http://tfs.cs.tu-berlin.de/agg`
16. Universität Karlsruhe: Graph Rewrite Generator, GrGen (2010), `http://www.info.uni-karlsruhe.de/software.php/id=7`
17. University of Twente: Graphs for Object-Oriented Verification (GROOVE) (2011), `http://groove.cs.utwente.nl/`